

Búsqueda Semántica en Español: Comparación de Representaciones Vectoriales de Texto

Alejandro Vásquez Sánchez
Proyecto Final — Inteligencia Artificial
May 28, 2026

1. Planteamiento del problema

Cuando usamos un buscador, esperamos que entienda lo que *queremos decir* y no solo las palabras exactas que escribimos. Un buscador tradicional que solo compara palabras literales falla cuando la pregunta y la respuesta usan términos distintos para hablar de lo mismo (por ejemplo, “computadora” y “ordenador”). Este proyecto aborda ese problema: construir un **buscador semántico** en español capaz de encontrar el documento más relevante para una pregunta, incluso cuando no comparten exactamente las mismas palabras.

El objetivo concreto fue **comparar cinco maneras distintas de representar texto como vectores numéricos** y medir cuál encuentra mejor los documentos relevantes. La idea central detrás de todas ellas es la misma: convertir cada palabra o documento en un vector de números, de modo que textos con significados parecidos queden “cerca” en el espacio matemático. Una vez que el texto es un vector, buscar se reduce a encontrar los documentos cuyos vectores más se parecen al de la pregunta.

Se usaron dos conjuntos de datos. Para **aprender** las representaciones de palabras se utilizó un subconjunto de la **Wikipedia en español** (50.000 artículos, unos 25 millones de palabras tras la limpieza). Para **evaluar** el buscador se utilizó **mMARCO**, la versión en español de un conjunto estándar de preguntas y pasajes relevantes; se trabajó con 5.000 documentos y 200 preguntas con su respuesta correcta conocida.

2. Modelos utilizados

Se compararon cinco enfoques, ordenados de más simple a más sofisticado:

- **TF-IDF** (referencia clásica): representa cada documento contando qué palabras contiene, dando más peso a las palabras poco comunes. No entiende sinónimos, solo coincidencias literales.
- **Word2Vec (implementado desde cero)**: el modelo central del proyecto. Se programó en PyTorch la arquitectura *Skip-gram*, que aprende el significado de cada palabra a partir de las palabras que la rodean en el texto. La idea es que “una palabra se conoce por la compañía que tiene”.
- **Word2Vec (con la librería gensim)**: el mismo modelo anterior, pero usando una implementación profesional ya existente. Sirve como punto de comparación para validar que nuestra versión casera funciona bien.
- **GloVe**: vectores de palabras preentrenados por la Universidad de Chile sobre un corpus gigante en español. Representa otra familia clásica de embeddings.
- **Sentence-Transformers (SBERT)**: un modelo moderno basado en redes Transformer (la misma tecnología detrás de los modelos de lenguaje actuales). A diferencia de los anteriores, entiende la oración completa en su contexto, no palabra por palabra.

Detalle de la arquitectura Skip-gram (modelo principal). El modelo aprende dos tablas de vectores, una para la palabra “central” y otra para las palabras de su “contexto”. Durante el entrenamiento, por cada palabra del texto el modelo intenta predecir qué palabras aparecen a su alrededor. Para que esto sea computacionalmente viable se usó la técnica de *negative sampling*, que en lugar de evaluar todo el vocabulario en cada paso, solo contrasta la respuesta correcta contra unas pocas palabras incorrectas elegidas al azar. También se aplicó *subsampling* de palabras muy frecuentes (como “de” o “la”), que aportan poco significado y dominarían el aprendizaje. Cada palabra se representó con un vector de 100 dimensiones. Para representar un documento completo con los modelos de palabra, se promediaron los vectores de todas sus palabras.

3. Procedimiento de entrenamiento

El único modelo entrenado desde cero fue el Word2Vec en PyTorch (los demás se usaron preentrenados o con librería). Sus parámetros fueron:

Parámetro	Valor	Parámetro	Valor
Optimizador	Adam	Función de pérdida	Entropía cruzada binaria
Tasa de aprendizaje	0.005	Tamaño de batch	4.096
Épocas	15	Dimensión del vector	100
Muestras negativas	5	Hardware	GPU Tesla T4 (Colab)

La función de pérdida (entropía cruzada binaria) mide qué tan bien el modelo distingue las palabras que realmente aparecen juntas de las que no. El entrenamiento tomó aproximadamente 4 horas en la GPU. La pérdida descendió de forma estable de 2,40 a 2,25 a lo largo de las 15 épocas, indicando una convergencia sana sin sobreajuste.

Una verificación cualitativa confirmó que el modelo aprendió relaciones reales. Las palabras más cercanas a “rey” resultaron ser “trono”, “reinado” y “monarca”; y las analogías clásicas funcionaron: al resolver “hombre es a rey como mujer es a _____”, el modelo respondió correctamente “reina”; y “Francia es a París como España es a _____” dio “Madrid”.

4. Resultados

El desempeño se midió con dos métricas estándar de buscadores. La **Precisión@K** indica con qué frecuencia el documento correcto aparece entre los primeros K resultados. El **MRR** (rango recíproco medio) premia que el documento correcto aparezca lo más arriba posible en la lista. En ambas, más alto es mejor.

Método	Precisión@1	Precisión@5	Precisión@10	MRR
TF-IDF (referencia)	54,0 %	79,0 %	82,5 %	64,6 %
Word2Vec (PyTorch, propio)	38,0 %	60,5 %	71,5 %	49,7 %
Word2Vec (gensim)	35,0 %	62,0 %	69,5 %	46,6 %
GloVe (preentrenado)	30,0 %	49,5 %	57,0 %	39,0 %
Sentence-Transformers	63,0 %	83,0 %	88,5 %	72,3 %

Interpretación. El modelo moderno **Sentence-Transformers fue el claro ganador**, acertando el documento correcto en el primer intento el 63 % de las veces. Esto era esperable: al entender la oración completa en contexto, capta el significado mejor que cualquier método basado en palabras sueltas.

Un resultado interesante es que **TF-IDF quedó en segundo lugar**, por encima de todos los Word2Vec y GloVe. Esto ocurre porque en mMARCO las preguntas y sus respuestas suelen compartir varias palabras literales, situación ideal para un método de coincidencia exacta. Muestra que un método simple, en el escenario adecuado, sigue siendo muy competitivo.

Entre los modelos de palabra, **nuestra implementación propia de Word2Vec superó ligeramente a la de gensim** (38 % vs 35 % en Precisión@1), lo que valida que la implementación desde cero fue correcta. Ambos superaron a GloVe; la razón más probable es que nuestro Word2Vec se entrenó sobre Wikipedia en español, más parecida al lenguaje de mMARCO, mientras que GloVe provenía de un corpus distinto. La principal limitación de estos tres es que, al promediar los vectores de todas las palabras de un documento, se diluye la información de las palabras clave.

La Figura ?? muestra visualmente que los embeddings aprendidos sí capturan significado: al proyectar las palabras a dos dimensiones, estas se agrupan espontáneamente por categorías (países, ciudades, días, meses, familia y animales), sin que nadie se lo indicara explícitamente al modelo.

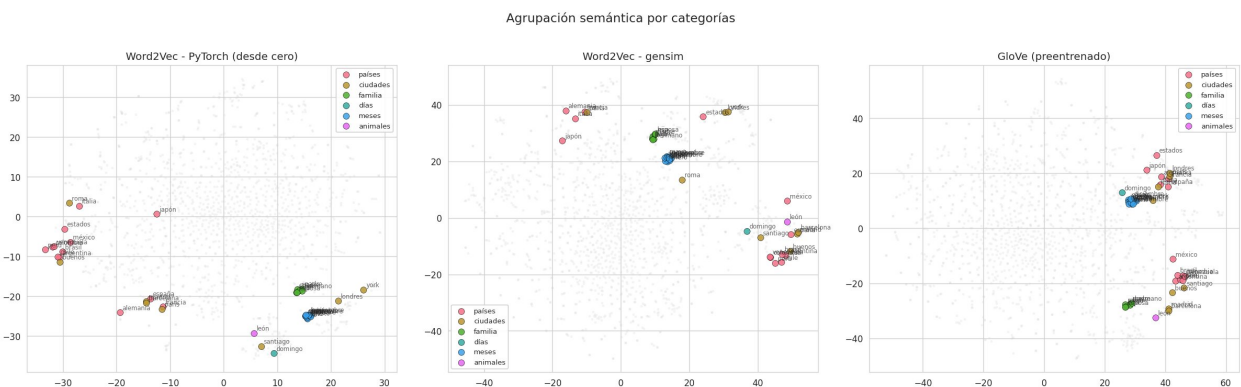


Figura 1: Proyección en dos dimensiones (t-SNE) de las palabras más frecuentes. Cada color es una categoría semántica. Que las palabras de una misma categoría aparezcan agrupadas confirma que los modelos aprendieron relaciones de significado, tanto en el Word2Vec propio como en gensim y GloVe.

Conclusión. El proyecto cumplió su objetivo: se implementó Word2Vec desde cero con resultados verificables y se comparó de forma justa contra alternativas clásicas y modernas. La lección principal es que no existe un único “mejor” método: los modelos modernos tipo Transformer ofrecen la mejor calidad semántica, pero técnicas simples como TF-IDF siguen siendo sorprendentemente eficaces cuando las preguntas y los documentos comparten vocabulario.